

Source at (0,0) and moves right and down

```
import java.util.Scanner;
import java.util.ArrayList;

public class RatInMaze {
    private static int[][] maze;
    private static ArrayList<String> paths;

    private static boolean isSafe(int row, int col, int n) {
        return (row >= 0 && row < n && col >= 0 && col < n &&
            maze[row][col] == 1);
    }

    private static void solve(int row, int col, int n, String path) {
        if (row == n-1 && col == n-1) {
            paths.add(path);
            return;
        }

        maze[row][col] = 0;

        // Try moving down
        if (isSafe(row + 1, col, n)) {
            solve(row + 1, col, n, path + "D");
        }

        // Try moving right
        if (isSafe(row, col + 1, n)) {
            solve(row, col + 1, n, path + "R");
        }

        // Backtrack: restore cell to original value
        maze[row][col] = 1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt(); // Read N (rows = cols since N*N)
        maze = new int[n][n];
        paths = new ArrayList<>();

        // Read N*N matrix
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                maze[i][j] = sc.nextInt();
            }
        }
    }
}
```

```

    }
}

// Check if start and end are valid
if (maze[0][0] == 1 && maze[n-1][n-1] == 1) {
    solve(0, 0, n, "");
}

// Print all paths (or "No Solution" if none)
if (paths.isEmpty()) {
    System.out.println("No Solution");
} else {
    for (String path : paths) {
        System.out.print(path + " ");
    }
    System.out.println();
}

sc.close();
}
}

```

Source at (n-1, n-1) and moves up and left

```

import java.util.Scanner;
import java.util.ArrayList;

public class RatInMazeReverse2 {
    private static int[][] maze;
    private static ArrayList<String> paths;

    private static boolean isSafe(int row, int col, int n) {
        return row >= 0 && row < n && col >= 0 && col < n && maze[row][col] == 1;
    }

    private static void solve(int row, int col, int n, String path) {
        if (row == 0 && col == 0) {
            paths.add(path);
            return;
        }

        maze[row][col] = 0;

        // Move up -> means Down in forward direction
        if (isSafe(row - 1, col, n)) {
            solve(row - 1, col, n, "D" + path);
        }

        // Move left -> means Right in forward direction

```

```

        if (isSafe(row, col - 1, n)) {
            solve(row, col - 1, n, "R" + path);
        }

        maze[row][col] = 1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        maze = new int[n][n];
        paths = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                maze[i][j] = sc.nextInt();
            }
        }

        if (maze[n - 1][n - 1] == 1 && maze[0][0] == 1) {
            solve(n - 1, n - 1, n, "");
        }

        if (paths.isEmpty()) {
            System.out.println("No Solution");
        } else {
            for (String p : paths) {
                System.out.print(p + " ");
            }
            System.out.println();
        }

        sc.close();
    }
}

```

Source at (0,0) and moves up and left

```

import java.util.Scanner;
import java.util.ArrayList;

public class RatInMazeReverse2 {
    private static int[][] maze;
    private static ArrayList<String> paths;

    private static boolean isSafe(int row, int col, int n) {
        return row >= 0 && row < n && col >= 0 && col < n && maze[row][col] == 1;
    }
}

```

```

private static void solve(int row, int col, int n, String path) {
    if (row == 0 && col == 0) {
        paths.add(path);
        return;
    }

    maze[row][col] = 0;

    // Move up -> means Down in forward direction
    if (isSafe(row - 1, col, n)) {
        solve(row - 1, col, n, "D" + path);
    }

    // Move left -> means Right in forward direction
    if (isSafe(row, col - 1, n)) {
        solve(row, col - 1, n, "R" + path);
    }

    maze[row][col] = 1;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();

    maze = new int[n][n];
    paths = new ArrayList<>();

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            maze[i][j] = sc.nextInt();
        }
    }

    if (maze[n - 1][n - 1] == 1 && maze[0][0] == 1) {
        solve(n - 1, n - 1, n, "");
    }

    if (paths.isEmpty()) {
        System.out.println("No Solution");
    } else {
        for (String p : paths) {
            System.out.print(p + " ");
        }
        System.out.println();
    }

    sc.close();
}

```

}

}